Examen de Programmation Orientée Objet

Nom & Prénom : _____	Date : _____

Lisez bien les consignes. <u>Le non-respect de celles-ci peut entraîner un refus.</u>
<u>Consignes :</u>

- Pas de documentation et/ou papiers autres que ceux distribués par l'enseignant.
- Pas de téléphone, tablette ou autres objets connectés (montres...).
- Pas d'accès Internet, sauf accès Teams pour le transfert du fichier final.
- La partie pratique se fait sur l'ordinateur qui vous est assigné par l'enseignant.
- Utilisation IA interdite (chatGPT, Copilot...).
- Les classes doivent être réutilisables (pas de lambdas !).

Créez une application type console avec le logiciel Eclipse en suivant scrupuleusement les étapes ci-dessous.

Nous allons modéliser les premières étapes d'un système de gestion de dresseurs Pokémon.

Un dresseur possède un identifiant unique, un nom, un prénom, une ville de départ et une catégorie (DÉBUTANT, CONFIRMÉ, CHAMPION). Chaque dresseur peut capturer des Pokémon. Un Pokémon est identifié par son numéro de Pokédex, un nom, un type principal (FEU, EAU, PLANTE, ELECTRIK, NORMAL, PSY), un niveau (entier entre 1 et 100) et un nombre de points de vie.

La « Map » est imposée comme structure de données pour les dresseurs et les Pokémon.

L'utilisateur doit pouvoir ajouter des dresseurs.

Un Pokémon doit pouvoir être ajouté à un dresseur déjà existant.

Une exception EquipeFullException doit être lancée si on essaie d'ajouter plus de Pokémon que le maximum autorisé par catégorie : DÉBUTANT → max 3, CONFIRMÉ → max 5, CHAMPION → max 6.

L'application doit pouvoir fournir le nombre total de Pokémon capturés (tous dresseurs confondus).

L'application doit pouvoir donner la liste de tous les Pokémon, triée par dresseur ou triée par nom de Pokémon.

L'application doit pouvoir donner la liste des Pokémon dont le niveau est inférieur à 10, triée par nom de Pokémon.

Créez un Singleton utilitaire qui imprime à la console les différents retours de méthodes. Commentez (JavaDoc) uniquement les éléments qui font de votre classe un Singleton.

Créez une classe Test (JUnit) et testez que votre Singleton est bien un Singleton (une seule méthode à tester).

<u>Barème indicatif :</u>

Élément évalué	**Points**
Modélisation : classes, enum, héritage, encapsulation	8

Exception EquipeFullException et gestion correcte	4
Structure Map et ajout de dresseurs / Pokémon	6
Méthodes de liste et de tri (par dresseur, par nom, niveau < 10)	8
Méthode comptage total Pokémon	2
Singleton (conception + JavaDoc)	6
Test JUnit (Singleton)	4
Qualité générale du code (nommage, encapsulation, compilation)	2
TOTAL	**40**

Déposez votre projet sous forme d'archive ZIP sur Teams avant la fin de l'épreuve. Tout projet qui ne compile pas sera refusé.